

03_relacion-rules-ingest-alerts

#python #api #fastapi #PostgreSQL #SQLAlchemy #backend #SIEM

1 Objetivo de la nota

Esta nota explica la relación técnica entre las reglas (`Rule`), la ingesta de eventos (`ingest.py`) y la generación de alertas (`Alert`) dentro del laboratorio SIEM MVP.

A diferencia de las notas anteriores, esta no analiza un único archivo línea por línea. Su función es unir varias piezas ya estudiadas para entender el flujo completo del motor de reglas.

Los archivos implicados son:

```
backend/app/schemas/rule.py
backend/app/api/routes/rules.py
backend/app/models/rule.py
backend/app/api/routes/ingest.py
backend/app/models/event.py
backend/app/models/alert.py
```

El objetivo principal es entender esta cadena:

```
RuleCreate
  ↓
Rule
  ↓
POST /ingest
  ↓
Event
  ↓
evaluación de reglas
  ↓
Alert
```

2 Archivos relacionados

```
backend/app/schemas/rule.py
```

Define cómo se validan los datos necesarios para crear una regla.

```
backend/app/api/routes/rules.py
```

Define los endpoints `POST /rules` y `GET /rules`.

```
backend/app/models/rule.py
```

Define el modelo ORM `Rule`, que representa la tabla `rules`.

```
backend/app/api/routes/ingest.py
```

Contiene la lógica que consulta reglas activas y evalúa eventos contra ellas.

```
backend/app/models/event.py
```

Define el modelo `Event`, que representa los eventos recibidos.

```
backend/app/models/alert.py
```

Define el modelo `Alert`, que representa las alertas generadas cuando una regla coincide.

3 Visión general del flujo

El motor de reglas funciona en dos momentos diferentes:

1. Configuración de reglas
2. Evaluación de reglas durante la ingesta

La primera parte ocurre cuando el usuario crea una regla mediante la API.

La segunda ocurre cuando llega un evento nuevo al endpoint `/ingest`.

Flujo completo:

```
POST /rules
  ↓
se crea una Rule
  ↓
la regla queda guardada en PostgreSQL
  ↓
POST /ingest
  ↓
llega un Event
  ↓
se consultan Rules activas
  ↓
se compara Event contra cada Rule
  ↓
si coincide, se crea una Alert
```

4 Fase 1: creación de una regla

La creación de reglas empieza en el endpoint:

```
POST /rules
```

Este endpoint está definido en:

```
backend/app/api/routes/rules.py
```

El cliente envía un JSON parecido a este:

```
{
  "name": "Failed login auth",
  "enabled": true,
  "source": "auth",
  "severity_min": 3,
  "contains": "failed login",
  "throttle_seconds": 300,
  "threshold_count": 5,
  "threshold_seconds": 60,
  "meta_match": {
    "user": "admin"
  }
}
```

Ese JSON se valida con el schema:

```
RuleCreate
```

definido en:

5 Validación con RuleCreate

El schema `RuleCreate` define qué campos puede enviar el usuario al crear una regla.

Campos principales:

```
name
enabled
source
severity_min
contains
throttle_seconds
threshold_count
threshold_seconds
meta_match
```

Este schema aplica validaciones antes de llegar a la base de datos.

Ejemplos:

```
name           → mínimo 1 carácter, máximo 120
source         → máximo 64 caracteres
severity_min    → entre 0 y 10
contains       → máximo 200 caracteres
throttle_seconds → entre 0 y 86400
threshold_count → entre 1 y 100000
threshold_seconds → entre 1 y 86400
```

Esto evita que entren reglas mal formadas.

La relación es:

```
JSON recibido
  ↓
RuleCreate
  ↓
datos validados
```

Si el JSON no cumple las restricciones, FastAPI devuelve error de validación y la regla no se crea.

6 Conversión de RuleCreate a Rule

Una vez validado el payload, `rules.py` crea un objeto `Rule` :

```
rule = Rule(
    name=payload.name,
    enabled=payload.enabled,
    source=payload.source,
    severity_min=payload.severity_min,
    contains=payload.contains,
    meta_match=payload.meta_match,
    throttle_seconds=payload.throttle_seconds,
    threshold_count=payload.threshold_count,
    threshold_seconds=payload.threshold_seconds,
)
```

Aquí ocurre el paso de schema Pydantic a modelo SQLAlchemy.

Relación campo a campo:

```
payload.name          → rule.name
payload.enabled       → rule.enabled
payload.source        → rule.source
payload.severity_min  → rule.severity_min
payload.contains      → rule.contains
payload.meta_match    → rule.meta_match
payload.throttle_seconds → rule.throttle_seconds
payload.threshold_count → rule.threshold_count
payload.threshold_seconds → rule.threshold_seconds
```

Conceptualmente:

```
RuleCreate
  ↓
Rule
```

`RuleCreate` valida datos.

`Rule` representa una fila real en la tabla `rules`.

7 Persistencia de la regla

Después de crear el objeto `Rule`, el endpoint lo añade a la sesión de base de datos:

```
db.add(rule)
```

Luego intenta confirmar la transacción:

```
db.commit()
```

Si todo va bien, la regla queda almacenada en PostgreSQL.

Relación:

```
Rule
  ↓
db.add(rule)
  ↓
db.commit()
  ↓
tabla rules
```

La tabla afectada es:

```
rules
```

8 Control de reglas duplicadas

El campo `name` de la regla es único.

En el modelo `Rule`, se define como:

```
name: Mapped[str] = mapped_column(String(120), nullable=False, unique=True)
```

Esto significa que no pueden existir dos reglas con el mismo nombre.

Si el usuario intenta crear una regla duplicada, SQLAlchemy puede lanzar:

```
IntegrityError
```

El endpoint lo captura así:

```
except IntegrityError:
    db.rollback()
    raise HTTPException(status_code=409, detail="Rule name already exists")
```

Esto devuelve un error HTTP:

```
409 Conflict
```

Con respuesta:

```
{
  "detail": "Rule name already exists"
}
```

Este control evita duplicidades y mantiene la tabla `rules` más limpia.

9 Fase 2: Llegada de un evento a `/ingest`

Una vez existen reglas en la base de datos, el flujo principal ocurre cuando llega un evento al endpoint:

```
POST /ingest
```

Este endpoint está definido en:

```
backend/app/api/routes/ingest.py
```

Ejemplo de evento recibido:

```
{
  "source": "auth",
  "severity": 4,
  "message": "Failed login attempt for user admin",
  "meta": {
    "host": "server-01",
    "user": "admin",
    "ip": "192.168.1.10"
  }
}
```

El evento se valida con:

```
IngestPayload
```

Después se crea un objeto:

```
Event
```

y se guarda en la base de datos.

10 Creación del evento

En `ingest.py`, el evento se crea así:

```
ev = Event(
    ts=now,
    source=payload.source,
    severity=payload.severity,
    message=payload.message,
```

```
    meta=payload.meta,  
)
```

Este objeto representa una fila de la tabla:

```
events
```

Después se añade a la sesión:

```
db.add(ev)
```

Y se ejecuta:

```
db.flush()
```

El `flush` es importante porque permite obtener:

```
ev.id
```

antes de hacer `commit`.

Esto es necesario porque, si una regla coincide, la alerta necesitará guardar:

```
event_id = ev.id
```

1.1 Cálculo de `group_key`

Después de crear el evento, `ingest.py` calcula una clave de agrupación:

```
group_key = _compute_group_key(ev)
```

La función auxiliar es:

```
def _compute_group_key(ev: Event) -> str | None:  
    if not ev.meta:  
        return None  
    return ev.meta.get("host")
```

Esto significa que el sistema busca:

```
meta.host
```

Ejemplo:

```
{  
  "meta": {  
    "host": "server-01"  
  }  
}
```

Resultado:

```
group_key = "server-01"
```

Si no existe `meta` o no existe `host`, el resultado será:

```
group_key = None
```

El `group_key` es importante porque afecta a:

```
throttle
anti-duplicado
threshold
agrupación de alertas
```

1 2 Consulta de reglas activas

Después de guardar temporalmente el evento, `ingest.py` consulta las reglas activas:

```
rules = db.execute(
    select(Rule).where(Rule.enabled.is_(True)).order_by(Rule.id.asc())
).scalars().all()
```

Esta consulta obtiene únicamente reglas donde:

```
enabled = true
```

Por tanto, una regla deshabilitada sigue existiendo en la tabla `rules`, pero no se evalúa.

La relación es:

```
tabla rules
  ↓
solo enabled = true
  ↓
lista de reglas activas
```

Ordena por:

```
Rule.id ascendente
```

Esto hace que las reglas se evalúen según el orden en que fueron creadas.

1 3 Evaluación de cada regla

Una vez obtenidas las reglas activas, el sistema las recorre:

```
for rule in rules:
```

Por cada regla, se comprueban varios criterios.

Si un criterio no se cumple, se ejecuta:

```
continue
```

Esto significa:

```
esta regla no aplica;
pasa a la siguiente regla
```

El flujo es:

```
rule 1
  ↓
¿coincide?
  ↓
sí/no

rule 2
  ↓
```

```
¿coincide?  
↓  
sí/no  
  
rule 3  
↓  
¿coincide?  
↓  
sí/no
```

Una misma ingesta puede generar cero, una o varias alertas, dependiendo de cuántas reglas coincidan.

1 4 Criterio source

El primer criterio es el origen:

```
if rule.source and ev.source != rule.source:  
    continue
```

Si la regla tiene definido `source`, el evento debe tener el mismo origen.

Ejemplo:

```
rule.source = "auth"  
ev.source   = "auth"
```

Coincide.

Ejemplo contrario:

```
rule.source = "auth"  
ev.source   = "firewall"
```

No coincide.

Si `rule.source` es `None`, este criterio no se aplica.

1 5 Criterio severity_min

El segundo criterio es la severidad mínima:

```
if rule.severity_min is not None and ev.severity < rule.severity_min:  
    continue
```

Ejemplo:

```
rule.severity_min = 5  
ev.severity       = 3
```

Como $3 < 5$, la regla no aplica.

Ejemplo válido:

```
rule.severity_min = 5  
ev.severity       = 7
```

Como $7 \geq 5$, la regla supera este criterio.

Si `severity_min` es `None`, no se filtra por severidad.

1 6 Criterio contains

El tercer criterio es la búsqueda de texto dentro del mensaje:

```
if rule.contains and rule.contains.lower() not in (ev.message or "").lower():
    continue
```

Ejemplo:

```
rule.contains = "failed login"
ev.message    = "Failed login attempt for user admin"
```

Coincide.

La comparación usa `.lower()` en ambos textos, por lo que no distingue mayúsculas y minúsculas.

Esto evita que fallen coincidencias por diferencias como:

```
Failed Login
failed login
FAILED LOGIN
```

17 Criterio `meta_match`

El cuarto criterio es la coincidencia exacta sobre metadatos:

```
if rule.meta_match:
    if not ev.meta:
        continue
    if any(ev.meta.get(k) != v for k, v in rule.meta_match.items()):
        continue
```

Si la regla tiene `meta_match`, el evento debe tener `meta`.

Después, cada clave y valor de `meta_match` debe coincidir con el `meta` del evento.

Ejemplo de regla:

```
{
  "meta_match": {
    "user": "admin",
    "facility": "auth"
  }
}
```

Evento válido:

```
{
  "meta": {
    "user": "admin",
    "facility": "auth",
    "host": "server-01"
  }
}
```

Evento no válido:

```
{
  "meta": {
    "user": "guest",
    "facility": "auth"
  }
}
```

No coincide porque:

```
user != admin
```

1 8 Resultado de los criterios básicos

Si el evento supera:

```
source
severity_min
contains
meta_match
```

entonces la regla se considera candidata para generar alerta.

Pero antes de crearla, el sistema puede aplicar controles adicionales:

```
throttle
anti-duplicado
threshold
```

Estos controles reducen ruido y evitan alertas repetidas o prematuras.

1 9 Throttle

El throttle limita la frecuencia con la que una regla puede generar alertas para el mismo grupo.

El código solo lo aplica si:

```
group_key is not None
and rule.throttle_seconds is not None
and rule.throttle_seconds > 0
```

Esto significa:

```
debe existir group_key
debe existir throttle_seconds
throttle_seconds debe ser mayor que 0
```

Si `group_key` es `None`, el throttle no se aplica.

La razón es que no hay una agrupación fiable.

2 0 Consulta de última alerta para throttle

Para aplicar throttle, el sistema busca la última alerta activa de la misma regla y grupo:

```
select(Alert.created_at)
.where(
    Alert.rule_id == rule.id,
    Alert.group_key == group_key,
    Alert.status.in_(("open", "ack")),
)
.order_by(Alert.created_at.desc())
.limit(1)
```

Esta consulta busca alertas:

```
de la misma regla
del mismo group_key
con estado open o ack
```

No considera alertas cerradas.

Esto tiene sentido porque una alerta cerrada ya no debería bloquear indefinidamente nuevas alertas.

2.1 Comparación temporal del throttle

Si existe una alerta anterior, se calcula la diferencia:

```
delta = (now - last_alert_ts).total_seconds()
```

Después se compara con el throttle:

```
if delta < rule.throttle_seconds:
    continue
```

Ejemplo:

```
rule.throttle_seconds = 300
última alerta hace 100 segundos
```

Resultado:

```
100 < 300
  ↓
no se crea alerta
```

Si han pasado más de 300 segundos, el flujo continúa.

2.2 Anti-duplicado

Después del throttle, el sistema aplica una comprobación anti-duplicado.

El objetivo es evitar crear otra alerta si ya existe una alerta activa para la misma regla y grupo.

Consulta:

```
select(Alert.id)
.where(
    Alert.rule_id == rule.id,
    Alert.group_key == group_key,
    Alert.status.in_(("open", "ack")),
)
.order_by(Alert.created_at.desc())
.limit(1)
```

Si devuelve un ID, significa que ya existe una alerta activa.

Entonces se ejecuta:

```
continue
```

y no se crea otra alerta.

2.3 Diferencia entre throttle y anti-duplicado

Throttle y anti-duplicado se parecen, pero no son lo mismo.

```
Throttle
  ↓
evita alertas demasiado frecuentes durante un tiempo definido
```

Anti-duplicado

↓

evita crear otra alerta si ya hay una abierta o reconocida

Ejemplo:

```
throttle_seconds = 300
```

Puede bloquear nuevas alertas durante 5 minutos.

El anti-duplicado puede bloquearlas mientras exista una alerta `open` o `ack`, aunque hayan pasado más de 5 minutos.

En este proyecto, ambos mecanismos reducen ruido.

El anti-duplicado es más fuerte porque depende del estado de la alerta.

2.4 Threshold

El threshold permite generar alerta solo si hay varios eventos coincidentes dentro de una ventana temporal.

Se aplica si existen ambos campos:

```
rule.threshold_count is not None
and rule.threshold_seconds is not None
```

Ejemplo de regla:

```
{
  "threshold_count": 5,
  "threshold_seconds": 60
}
```

Significa:

5 eventos en 60 segundos

2.5 Threshold requiere group_key

El código exige:

```
if group_key is None:
    continue
```

Por tanto, si no existe `meta.host`, no se aplica threshold.

Esto es importante.

Para que una regla con threshold funcione correctamente, los eventos deben incluir:

```
{
  "meta": {
    "host": "server-01"
  }
}
```

Así el sistema puede contar eventos agrupados por host.

2.6 Ventana temporal del threshold

El inicio de la ventana se calcula así:

```
window_start = now - timedelta(seconds=rule.threshold_seconds)
```

Ejemplo:

```
now = 12:00:00
threshold_seconds = 60
window_start = 11:59:00
```

Después se cuentan eventos desde ese momento:

```
stmt = select(func.count(Event.id)).where(Event.ts >= window_start)
```

Esto permite responder a la pregunta:

```
¿Cuántos eventos coincidentes han ocurrido en los últimos X segundos?
```

2.7 Filtros usados en threshold

La consulta del threshold reutiliza los criterios de la regla:

```
source
severity_min
contains
meta_match
host/group_key
```

Filtros aplicados:

```
if rule.source:
    stmt = stmt.where(Event.source == rule.source)
```

```
if rule.severity_min is not None:
    stmt = stmt.where(Event.severity >= rule.severity_min)
```

```
if rule.contains:
    stmt = stmt.where(Event.message.ilike(f"%{rule.contains}%"))
```

```
if rule.meta_match:
    stmt = stmt.where(Event.meta.contains(rule.meta_match))
```

```
stmt = stmt.where(Event.meta.contains({"host": group_key}))
```

Esto hace que el conteo no sea genérico, sino ajustado a la regla actual.

2.8 Comparación con threshold_count

El número de eventos encontrados se guarda en:

```
matched_count = db.execute(stmt).scalar_one()
```

Luego se compara:

```
if matched_count < rule.threshold_count:
    continue
```

Ejemplo:

```
threshold_count = 5
matched_count = 4
```

No se genera alerta.

Ejemplo:

```
threshold_count = 5
matched_count = 5
```

Sí puede generarse alerta.

2.9 Creación de la alerta

Si el evento supera todos los criterios, se crea una alerta:

```
alert = Alert(
    rule_id=rule.id,
    event_id=ev.id,
    title=f"Rule matched: {rule.name}",
    group_key=group_key,
)
```

Relación:

```
rule.id → alert.rule_id
ev.id   → alert.event_id
```

Esto conecta la alerta con:

```
la regla que coincidió
el evento que la disparó
```

Después se añade a la sesión:

```
db.add(alert)
```

La alerta se confirmará cuando se ejecute:

```
db.commit()
```

3.0 Transacción completa

En `ingest.py`, el evento y las alertas se guardan dentro de la misma transacción.

Flujo:

```
db.add(ev)
db.flush()
db.add(alert)
db.commit()
```

Esto significa que la ingesta intenta guardar todo junto.

Si algo falla, se ejecuta:

```
db.rollback()
```

y se revierte la operación.

Esto evita estados intermedios incorrectos.

3.1 Flujo completo resumido

1. Se crea una regla con POST /rules.
2. La regla se valida con RuleCreate.
3. La regla se guarda como Rule en PostgreSQL.
4. Llega un evento a POST /ingest.
5. El evento se valida con IngestPayload.
6. Se crea un objeto Event.
7. Se guarda temporalmente con db.flush().
8. Se calcula group_key desde meta.host.
9. Se consultan reglas activas.
10. Cada Rule se compara con el Event.
11. Si no coincide, se salta con continue.
12. Si coincide, se aplican throttle, anti-duplicado y threshold.
13. Si todo se cumple, se crea Alert.
14. Se ejecuta db.commit().
15. La alerta queda disponible para consulta.

3 2 Diagrama técnico global

```
POST /rules
  ↓
RuleCreate
  ↓
Rule
  ↓
rules table
  ↓
POST /ingest
  ↓
IngestPayload
  ↓
Event
  ↓
events table
  ↓
select active Rule
  ↓
match source/severity/contains/meta
  ↓
throttle / anti-duplicado / threshold
  ↓
Alert
  ↓
alerts table
```

3 3 Relación con endpoints del laboratorio

La relación entre endpoints es:

```
POST /rules
  ↓
crea reglas

GET /rules
  ↓
permite revisar reglas creadas

POST /ingest
  ↓
usa reglas activas para evaluar eventos

GET /events
```

↓
permite revisar eventos recibidos

GET /alerts

↓
permite revisar alertas generadas

GET /metrics

↓
resume eventos, reglas y alertas

El motor de reglas no es una ruta aislada. Es una lógica transversal que conecta varias partes del backend.

3 4 Ejemplo práctico completo

1. Crear regla

```
curl -X POST http://localhost:8000/rules \  
-H "Content-Type: application/json" \  
-d '{  
  "name": "Failed login auth",  
  "enabled": true,  
  "source": "auth",  
  "severity_min": 3,  
  "contains": "failed login",  
  "meta_match": {  
    "user": "admin"  
  }  
}'
```

Esta regla significa:

Detectar eventos de origen auth,
con severidad mínima 3,
cuyo mensaje contenga failed login,
y cuyo meta.user sea admin.

2. Enviar evento coincidente

```
curl -X POST http://localhost:8000/ingest \  
-H "Content-Type: application/json" \  
-d '{  
  "source": "auth",  
  "severity": 4,  
  "message": "Failed login attempt for user admin",  
  "meta": {  
    "host": "server-01",  
    "user": "admin"  
  }  
}'
```

Este evento coincide porque:

source = auth	→ coincide
severity = 4 >= 3	→ coincide
message contiene failed login	→ coincide
meta.user = admin	→ coincide

Resultado esperado:


```
se crea Event
se evalúa Rule
se crea Alert
```

3. Consultar alertas

```
curl http://localhost:8000/alerts
```

Debería aparecer una alerta relacionada con la regla creada.

3 5 Puntos importantes

Las reglas se configuran antes de ingestar eventos

Si no hay reglas creadas, `/ingest` guardará eventos, pero no generará alertas.

Solo se evalúan reglas activas

Una regla con:

```
enabled = false
```

no participa en la evaluación.

Una regla puede coincidir con muchos eventos

Cada vez que llega un evento, se vuelve a evaluar contra las reglas activas.

Un evento puede activar varias reglas

Si varias reglas coinciden con el mismo evento, pueden generarse varias alertas.

`group_key` es clave para lógica avanzada

Sin `group_key`, no se aplican correctamente:

```
throttle
anti-duplicado
threshold
```

Por eso es importante enviar eventos con:

```
{
  "meta": {
    "host": "...
  }
}
```

Threshold requiere eventos acumulados

Una regla con `threshold` no tiene por qué generar alerta con el primer evento.

Necesita alcanzar el número configurado dentro de la ventana temporal.

El anti-duplicado depende del estado de la alerta

Si ya hay una alerta:

```
open
```

o:

```
ack
```

para la misma regla y grupo, no se crea otra.

Si la alerta está:

```
closed
```

sí puede generarse una nueva.

3.6 Comandos útiles relacionados

Listar reglas:

```
curl http://localhost:8000/rules
```

Crear regla simple:

```
curl -X POST http://localhost:8000/rules \
-H "Content-Type: application/json" \
-d '{
  "name": "High severity events",
  "enabled": true,
  "severity_min": 5
}'
```

Enviar evento de prueba:

```
curl -X POST http://localhost:8000/ingest \
-H "Content-Type: application/json" \
-d '{
  "source": "auth",
  "severity": 5,
  "message": "High severity authentication event",
  "meta": {
    "host": "server-01"
  }
}'
```

Consultar eventos:

```
curl http://localhost:8000/events
```

Consultar alertas:

```
curl http://localhost:8000/alerts
```

Consultar métricas:

```
curl http://localhost:8000/metrics
```

Consultar reglas en PostgreSQL:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT id, name, enabled, source, severity_min,
```

```
contains, throttle_seconds, threshold_count, threshold_seconds, meta_match FROM rules ORDER BY id DESC;"
```

Consultar eventos recientes:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT id, source, severity, message, meta, created_at  
FROM events ORDER BY id DESC LIMIT 10;"
```

Consultar alertas recientes:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT id, rule_id, event_id, title, group_key, status,  
created_at FROM alerts ORDER BY id DESC LIMIT 10;"
```

Consultar alertas con JOIN:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT a.id, a.title, a.status, r.name AS rule_name,  
e.source, e.severity, e.message FROM alerts a JOIN rules r ON a.rule_id = r.id JOIN events e ON  
a.event_id = e.id ORDER BY a.id DESC LIMIT 10;"
```

37 Resumen técnico

El motor de reglas permite transformar eventos almacenados en alertas operativas.

La creación de reglas se realiza mediante `POST /rules`, donde el payload se valida con `RuleCreate` y se guarda como modelo `Rule` en PostgreSQL.

Durante la ingesta, `POST /ingest` crea un evento `Event`, consulta las reglas activas y evalúa criterios como `source`, `severity_min`, `contains` y `meta_match`.

Si la regla coincide, el sistema aplica controles adicionales como `throttle`, anti-duplicado y `threshold`. Si todos los criterios se cumplen, se crea una alerta `Alert` vinculada al evento y a la regla.

La relación final es:

```
Rule + Event → Alert
```

Esta es una de las partes más importantes del laboratorio, porque convierte el proyecto de un simple almacén de eventos en un sistema básico de detección.